

Started on	Thursday, 12 June 2025, 12:47 AM
State	Finished
Completed on	Thursday, 12 June 2025, 8:45 AM
Time taken	7 hours 58 mins
Marks	2.00/2.00
Grade	10.00 out of 10.00 (100%)

Microsoft Excel là chương trình xử lý bảng tính nằm trong bộ Microsoft Office của hãng phần mềm Microsoft. Excel được thiết kế để giúp ghi lại, trình bày các thông tin xử lý dưới dạng bảng, thực hiện tính toán và xây dựng các số liệu thống kê trực quan.

Bảng tính Excel được chia thành nhiều hàng và cột. Giao điểm của hàng và cột gọi là một ô. Trong một ô của Excel, ta có thể nhập một giá trị hoặc một công thức có sử dụng giá trị của 1 hay nhiều ô khác. Khi ô A1 sử dụng giá trị của ô B2 ta nói A1 **tham chiếu đến** B2 hay A1 **phụ thuộc vào** B2.

Một lỗi thường gặp trong Excel là **tham chiếu vòng** (a circular reference). Tham chiếu vòng là trường hợp một ô tham chiếu đến chính nó hoặc như ví dụ bên dưới: A9 tham chiếu đến C2, C2 tham chiếu đến E9 và E9 tham chiếu đến A9.

Vấn đề đặt ra là làm thế nào để phát hiện có tham chiếu vòng trong bảng tính. Giả sử bảng tính có n ô, để đơn giản, ta đánh số các ô đang xét là 1, 2, 3, ..., n . Bạn biết được ô nào tham chiếu đến (các) ô nào. Hãy viết chương trình kiểm tra xem có xuất hiện tham chiếu vòng hay không.

Đầu vào (Input)

Dữ liệu đầu vào được nhập từ bàn phím với định dạng:

- Dòng đầu tiên chứa 2 số nguyên n và m , tương ứng là số ô và số tham chiếu.
- m dòng tiếp theo mỗi dòng chứa 2 số nguyên u, v nói rằng ô u tham chiếu đến ô v .

Đầu ra (Output)

In ra màn hình **CIRCULAR REFERENCE** nếu bảng tính có tham chiếu vòng, ngược lại in ra **OK**.

Chú ý

- Để chạy thử chương trình, bạn nên tạo một tập tin **dt.txt** chứa đồ thị cần kiểm tra.
- Thêm dòng **freopen("dt.txt", "r", stdin);** vào ngay sau hàm main(). Khi nộp bài, nhớ gỡ bỏ dòng này ra.
- Có thể sử dụng đoạn chương trình đọc dữ liệu mẫu sau đây:

```
freopen("dt.txt", "r", stdin); //Khi nộp bài nhớ bỏ dòng này.
Graph G;
int n, m, u, v, e;
scanf("%d%d", &n, &m);
init_graph(&G, n);

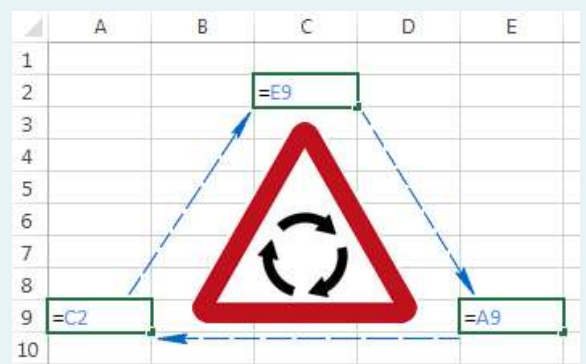
for (e = 0; e < m; e++) {
    scanf("%d%d", &u, &v);
    add_edge(&G, u, v, w);
}
```

For example:

Input	Result
3 3 1 3 2 1 3 2	CIRCULAR REFERENCE
7 9 1 2 1 3 1 4 2 3 2 6 3 5 3 7 4 5 5 7	OK

Answer: (penalty regime: 10, 20, ... %)

1 `#include <stdio.h>`



```

2
3 #define MAX_SIZE 100
4 typedef int ElementType;
5
6 //-----Graph-----
7 typedef struct{
8     int n,m;
9     int A[MAX_SIZE][MAX_SIZE];
10 }Graph;
11
12 //init graph with n vertex
13 void init_graph(Graph* pG, ElementType n){
14     pG->n = n;
15     pG->m = 0;
16
17     //cho toan bo ma tran ke bang 0
18     for(int i = 0; i <= pG->n; i++){
19         for(int j = 0; j <= pG->n; j++){
20             pG->A[i][j] = 0;
21         }
22     }
23 }
24
25 //add edge (u,v) to graph
26 void add_edge(Graph* pG, ElementType u, ElementType v){
27     pG->A[u][v] = 1;
28     // pG->A[v][u] = 1;
29     pG->m++;
30 }
31
32 //-----DFS-----
33 #define WHITE 0
34 #define GRAY 1
35 #define BLACK 2
36
37 int color[MAX_SIZE];
38 int has_cycle;
39
40 int visited[MAX_SIZE];
41 int parent[MAX_SIZE];
42
43 int adj(Graph* pG, int u, int v){
44     return pG->A[u][v] > 0;
45 }
46 void DFS (Graph* pG, int s, int p){
47     //1. to mau
48     color[s] = GRAY;
49
50     // visited[s] = 1;
51     // printf("%d \n", s);
52     parent[s] = p;
53     for(int u = 1; u <= pG->n; u++){
54         if(adj(pG,s,u)){
55             if(color[u] == WHITE){
56                 DFS(pG,u,s);
57             } else if(color[u] == GRAY){
58                 has_cycle = 1;
59             }
60         }
61     }
62     color[s] = BLACK;
63 }
64
65 int main(){
66     Graph G;
67     int n, m, u, v, e;
68     scanf("%d%d", &n, &m);
69     init_graph(&G, n);
70
71     for (e = 0; e < m; e++) {
72         scanf("%d%d", &u, &v);
73         add_edge(&G, u, v);
74     }
75     has_cycle = 0;
76     for(int i = 0; i <= G.n; i++){
77         visited[i] = 0;
78         parent[i] = -1;
79         color[i] = WHITE;

```

```

80     }
81
82     for(int i = 1; i <= G.n; i++){
83         if(!visited[i]){
84             DFS(&G,i,-1);
85         }
86     }
87
88     if(has_cycle){
89         printf("CIRCULAR REFERENCE");
90     }else{
91         printf("OK");
92     }
93
94 }

```

	Input	Expected	Got	
✓	3 3 1 3 2 1 3 2	CIRCULAR REFERENCE	CIRCULAR REFERENCE	✓
✓	7 9 1 2 1 3 1 4 2 3 2 6 3 5 3 7 4 5 5 7	OK	OK	✓
✓	7 9 1 2 1 3 2 4 2 5 2 6 3 5 3 6 4 7 5 7	OK	OK	✓
✓	3 3 1 2 2 3 3 1	CIRCULAR REFERENCE	CIRCULAR REFERENCE	✓
✓	4 3 1 2 2 3 2 4	OK	OK	✓
✓	7 9 4 5 1 2 1 4 2 3 2 6 7 3 3 1 5 7 7 6	CIRCULAR REFERENCE	CIRCULAR REFERENCE	✓

	Input	Expected	Got	
✓	5 5 1 2 2 3 3 4 4 5 5 1	CIRCULAR REFERENCE	CIRCULAR REFERENCE	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Thuyền trưởng Haddock (truyện Tintin) là một người luôn say xỉn. Vì thế đôi khi Tintin không biết ông ta đang say hay tỉnh. Một ngày nọ, Tintin hỏi ông ta về cách uống. Haddock nói như thế này: Có nhiều loại thức uống (soda, wine, water, ...), tuy nhiên Haddock lại tuân theo quy tắc *"để uống một loại thức uống nào đó cần phải uống tất cả các loại thức uống tiên quyết của nó"*. Ví dụ: để uống rượu (wine), Haddock cần phải uống soda và nước (water) trước. Vì thế muốn say cũng không phải dễ!

Cho danh sách các thức uống và các thức uống tiên quyết của nó. Hãy xét xem Haddock có thể nào say không ? Để làm cho Haddock say, ông ta phải uống hết tất cả các thức uống.

Ví dụ 1:

```
soda wine
water wine
```

Thức uống tiên quyết được cho dưới dạng a b, có nghĩa là để uống b bạn phải uống a trước. Trong ví dụ trên để uống wine, Haddock phải uống soda và water trước. Soda và water không có thức uống tiên quyết nên Haddock sẽ SAY.

Ví dụ 2:

```
soda wine
water wine
wine water
```

Để uống wine, cần uống water. Tuy nhiên để uống water lại cần wine. Vì thế Haddock không thể uống hết được các thức uống nên ông ta KHÔNG SAY.

Để đơn giản ta có thể giả sử các thức uống được mã hoá thành các số nguyên từ 1, 2, ... và dữ liệu đầu vào được cho có dạng như sau (ví dụ 1):

```
3 2
1 2
3 2
```

Có loại thức uống (soda: 1, wine: 2 và water: 3) và có 2 điều kiện tiên quyết

1 -> 2 và 3 -> 2.

Với ví dụ 2, ta có dữ liệu:

```
3 3
1 2
3 2
2 3
```

Viết chương trình đọc dữ liệu các thức uống và kiểm tra xem Haddock có thể say không. Nếu có in ra "YES", ngược lại in ra "NO".

Đầu vào (Input):

Dữ liệu đầu vào được nhập từ bàn phím (stdin) với định dạng:

- Dòng đầu tiên chứa 2 số nguyên **n** và **m**, tương ứng là số thức uống và số điều kiện tiên quyết
- **m** dòng tiếp theo mỗi dòng chứa 2 số nguyên **u v** nói rằng thức uống **u** là tiên quyết của thức uống **v**

Đầu ra (Output):

- Nếu Haddock có thể say in ra YES, ngược lại in ra NO.

Xem thêm các ví dụ bên dưới.

For example:

Input	Result
3 2 1 2 3 2	YES
3 3 1 2 3 2 2 3	NO



Input	Result
5 5	NO
1 2	
2 3	
3 4	
4 2	
5 4	

Answer: (penalty regime: 10, 20, ... %)

```

1  #include <stdio.h>
2
3  #define MAX_SIZE 100
4  typedef int ElementType;
5
6  //-----Graph-----
7  typedef struct{
8      int n,m;
9      int A[MAX_SIZE][MAX_SIZE];
10 }Graph;
11
12 //init graph with n vertex
13 void init_graph(Graph* pG, ElementType n){
14     pG->n = n;
15     pG->m = 0;
16
17     //cho toan bo ma tran ke bang 0
18     for(int i = 0; i <= pG->n; i++){
19         for(int j = 0; j <= pG->n; j++){
20             pG->A[i][j] = 0;
21         }
22     }
23 }
24
25 //add edge (u,v) to graph
26 void add_edge(Graph* pG, ElementType u, ElementType v){
27     pG->A[u][v] = 1;
28     // pG->A[v][u] = 1;
29     pG->m++;
30 }
31
32 //-----DFS-----
33 #define WHITE 0
34 #define GRAY 1
35 #define BLACK 2
36
37 int color[MAX_SIZE];
38 int has_cycle;
39
40 int visited[MAX_SIZE];
41 int parent[MAX_SIZE];
42
43 int adj(Graph* pG, int u, int v){
44     return pG->A[u][v] > 0;
45 }
46 void DFS (Graph* pG, int s, int p){
47     //1. to mau
48     color[s] = GRAY;
49
50     // visited[s] = 1;
51     // printf("%d \n", s);
52     parent[s] = p;
53     for(int u = 1; u <= pG->n; u++){
54         if(adj(pG,s,u)){
55             if(color[u] == WHITE){
56                 DFS(pG,u,s);
57             } else if(color[u] == GRAY){
58                 has_cycle = 1;
59             }
60         }
61     }
62     color[s] = BLACK;
63 }
64
65 int main(){

```

```

66 Graph G;
67 int n, m, u, v, e;
68 scanf("%d%d", &n, &m);
69 init_graph(&G, n);
70
71 for (e = 0; e < m; e++) {
72     scanf("%d%d", &u, &v);
73     add_edge(&G, u, v);
74 }
75 has_cycle = 0;
76 for(int i = 0; i <= G.n; i++){
77     visited[i] = 0;
78     parent[i] = -1;
79     color[i] = WHITE;
80 }
81
82 for(int i = 1; i <= G.n; i++){
83     if(!visited[i]){
84         DFS(&G,i,-1);
85     }
86 }
87
88 if(has_cycle){
89     printf("NO");
90 }else{
91     printf("YES");
92 }
93
94 }

```

	Input	Expected	Got	
✓	3 2 1 2 3 2	YES	YES	✓
✓	3 3 1 2 3 2 2 3	NO	NO	✓
✓	5 5 1 2 2 3 3 4 4 2 5 4	NO	NO	✓
✓	3 3 1 3 2 1 3 2	NO	NO	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

◀ * Bài tập 10. Kiểm tra đồ thị chứa chu trình

Jump to...

* Bài tập 12 - Kiểm tra đồ thị phân đôi ▶

